# 1. Introduction and Theory

A Generative Adversarial Network (GAN) is a deep-learning framework containing two neural networks that learn by competing. The Generator (G) creates synthetic samples. The Discriminator (D) estimates whether a sample is real (from the training dataset) or generated. As training proceeds, G tries to produce samples that resemble the training distribution, while D learns to identify generated samples.

## 1.1 Why GANs matter in cyber security

- Offensive / red-team research: researchers can study whether small changes in a feature representation cause a detector to misclassify a sample. This is useful for controlled robustness testing, but can also be misused to evade security tools.
- Defensive data augmentation: GANs can generate additional samples for rare attack categories, helping a classifier learn patterns that are under-represented in the original dataset.
- Anomaly detection and privacy research: GANs may model normal behaviour or create statistically similar datasets for experiments. Synthetic data must still be checked for leakage and realism.

## 1.2 Basic GAN objective

The original GAN objective is a minimax game: min_G max_D V(D,G) = E[x~pdata] log D(x) + E[z~pz] log(1 − D(G(z))). In simple terms, D wants real samples to score near 1 and generated samples near 0; G wants generated samples to score as real.

## 1.3 Generator architecture (conceptual)

**Random noise z → Dense layer → ReLU / Batch Normalization → Dense output layer → Synthetic feature vector x_fake**

Input: random latent vector (and optionally a target class condition). Hidden layers learn relationships among features. Output: a synthetic record in the same feature space as the training data. For tabular IDS data, continuous and categorical columns need suitable encoding or a tabular GAN such as CTGAN.

## 1.4 Discriminator architecture (conceptual)

**Real record x OR generated record x_fake → Dense layer → LeakyReLU / Dropout → Dense layer → Sigmoid score: real vs generated**

The discriminator is trained using real and generated examples. During the Generator update, its feedback guides G toward more realistic samples. The discriminator's real/fake score is not the final IDS decision; a separate downstream classifier usually predicts benign or attack labels.

## 1.5 Typical defensive pipeline

Collect authorised flow records → clean and encode features → split into train/validation/test sets → train GAN on training data (often a minority class) → generate synthetic training records → combine original and synthetic training data → train IDS classifier → evaluate once on an untouched real test set.

# 2. Part A — Offensive Application: MalGAN

## 2.1 How adversarial examples can evade a classifier

An adversarial example is an input that has been altered in a way that changes a model's prediction, while retaining the relevant properties of the underlying object. A security detector may rely on static features, behavioural features, or network-flow statistics. If the model has learned incomplete or brittle patterns, an adversarially selected feature representation may be assigned a benign label.

In a malware setting, the key constraint is that a modified file must remain functional and preserve its malicious behaviour; changing an abstract feature vector alone does not prove that a valid executable can be produced. In an IDS setting, generated traffic must obey protocol and flow constraints. This is why feature-space results should not automatically be treated as successful real-world attacks.

## 2.2 MalGAN methodology (paper summary)

Hu and Tan introduced MalGAN in "Generating Adversarial Malware Examples for Black-Box Attacks Based on GAN" (2017). Their setting assumes the attacker cannot inspect the target detector's internal parameters. The method uses a substitute detector to approximate the target's decisions, then trains a generator to create feature representations that receive lower malicious scores from the substitute. The generator and substitute detector are trained in an adversarial loop.

- Input: malware feature vectors and feedback from a target / substitute detection setup.
- Substitute detector: approximates the black-box detector's decision behaviour.
- Generator: learns to produce modified feature vectors intended to be classified as benign.
- Evaluation: measures whether the generated feature vectors evade the detector and how retraining-based defences respond.

Reported finding: the paper reports that MalGAN reduced the target detector's detection rate to nearly zero in its experimental setting. This is a result from that paper, not a guarantee that current commercial products or functional malware can be bypassed. The work demonstrates a risk of relying on a single learned detector and motivates robustness testing.

## 2.3 Ethical and practical concerns

- Dual use: methods can improve defensive testing but can also help malware authors evade detection.
- Validity: feature-space evasion does not necessarily create a valid, functioning malicious executable.
- Generalisation: success against one detector or dataset may not transfer to other detectors or current systems.
- Data and privacy: malware samples and telemetry can contain sensitive information; store and share them under approved controls.
- Responsible practice: use public benchmark data, isolated lab environments, explicit permission, and defensive reporting. Do not test against third-party systems or deploy generated malware.

# 3. Part B — Defensive Application: GAN-Based Data Augmentation

## 3.1 Problem: imbalanced IDS datasets

Intrusion-detection datasets often contain many benign or common attack records but relatively few examples of rare attacks. A classifier can obtain high overall accuracy while missing the rare class. Precision, recall, F1-score, confusion matrices, and per-class results are therefore important alongside accuracy.

## 3.2 CTGAN-based augmentation

CTGAN (Conditional Tabular GAN) is designed for mixed-type tabular data. It models continuous and categorical columns and can condition generation on a chosen category. In an IDS study, the training process can target an under-represented attack label, generate additional synthetic rows, and train a downstream classifier on the combined training data.

## 3.3 Defensive architecture diagram

| **Authorised labelled network-flow dataset** |
|---|
| Pre-processing: remove invalid rows, encode categories, scale numeric fields |
| GAN training on training split (optionally condition on minority attack class) |
| Generator → synthetic training records → quality / constraint checks |
| **Original training rows + accepted synthetic rows → IDS classifier → untouched real test set** |

The GAN is not itself necessarily the final intrusion detector. It creates candidate training data; the downstream classifier is evaluated on real, held-out records.

## 3.4 Evaluation protocol

- Use a fixed train/test split (or stratified split) and keep the test set real and untouched.
- Fit encoders, scalers, and the GAN using training data only to avoid data leakage.
- Train the same classifier twice: (A) original training set; (B) original plus synthetic training records.
- Report accuracy, precision, recall, macro-F1, per-class recall, and confusion matrix for both models.
- Repeat with fixed random seeds if possible. Report the change in percentage points: augmented accuracy − baseline accuracy.
- Check synthetic data for duplicates, impossible feature combinations, and overly similar records. Do not assume that more synthetic rows always improve generalisation.

## 3.5 Architecture diagram: Generator and Discriminator in tabular GAN

| **GENERATOR** | **DISCRIMINATOR** |
|---|---|
| Noise z + optional class condition<br>↓<br>Fully connected layers<br>↓<br>Output columns (numeric + categorical)<br>↓<br>Synthetic traffic row | Real row or generated row<br>↓<br>Encoded feature vector<br>↓<br>Fully connected layers<br>↓<br>Real / synthetic probability |

# 4. Results and Discussion

## 4.1 Published evidence

A study published in IEEE Access in 2022 evaluated CTGAN-based balancing on NSL-KDD and UNSW-NB15. Its abstract reports that training classifiers on CTGAN-balanced data increased prediction accuracy by up to 8% and improved Matthews correlation coefficient by up to 13% compared with imbalanced original data. These are maximum reported improvements across the study's experiments, not a universal expected gain.

A 2025 IEEE NetSoft study using a tailored CTGAN on NF-v2-Bot-IoT reported that the detection rate for a rare "theft" class increased from 42% to 99%. This is a class-specific detection-rate result, not overall accuracy, and should be labelled accordingly.

These studies support the hypothesis that synthetic minority-class data may improve rare-attack detection. However, the result depends on dataset quality, class imbalance, synthetic sample quality, classifier choice, and evaluation design.

## 4.2 Editable results table — fill after running your experiment

| Training setup | Accuracy (%) | Precision (%) | Recall (%) | Macro-F1 (%) |
|---|---|---|---|---|
| Original training data | _____ | _____ | _____ | _____ |
| Original + GAN samples | _____ | _____ | _____ | _____ |
| Change (percentage points) | _____ | _____ | _____ | _____ |

Do not fill the table with guessed values. If you are submitting a literature-analysis version rather than running code, replace the blanks with a clearly labelled published result and cite its source.

## 4.3 Interpretation template

The baseline classifier achieved _____% accuracy and _____ macro-F1. After adding GAN-generated training samples, accuracy was _____% and macro-F1 was _____. The accuracy changed by _____ percentage points. The recall for the rare _____ attack class changed from _____% to _____%. This suggests that augmentation [helped / did not help] under the selected setup. Possible explanations include [better minority-class coverage / synthetic noise / class overlap / overfitting].

## 4.4 Limitations

- GAN training can be unstable and may suffer from mode collapse, producing insufficient diversity.
- Synthetic samples can contain unrealistic feature combinations or reproduce bias from the source dataset.
- High accuracy can hide poor rare-class recall; multiple metrics and an untouched test set are required.
- Results from one dataset should not be generalised automatically to a live network.